Ejemplo sobre convención de llamada para C en X86-64

Arquitectura del Computador - LCC - FCEIA-UNR

3 de noviembre de 2026

Este ejemplo tiene como objetivo ilustrar algunos aspectos de la convención de llamada en C para la arquitectura x86-64. Además, resulta útil para familiarizarse con GDB.

Dado el siguiente código en lenguaje C, ubicado en el archivo `suma_larga.c`:

```
#include <stdio.h>
int suma(int a, int b, int c, int d, int e, int f, int g, int h){
    return a+b+c+d+e+f+g+h;
}

int main()
{
    printf("La suma es: %d\n", suma(1,2,3,4,5,6,7,8));
    return 0;
}
```

Compilar utilizando la opción `-g` para poder *debuggear* utilizando GDB:

```
$ gcc -g suma_larga.c
```

Luego ejecutar utilizando GDB:

```
$gdb ./a.out
```

Una vez que estamos dentro de la sesión de *debugging*, ponemos un *breakpoint* en `main` y ejecutamos el comando `run`:

```
(gdb) br main
Breakpoint 1 at 0x401166: file suma_larga.c, line 8.
(gdb) r
Starting program: /home/dferoldi/2025/a.out

Breakpoint 1, main () at suma_larga.c:8
```

Luego entramos a la función `suma` con el comando `step`:

```
(gdb) s
suma (a=1, b=2, c=3, d=4, e=5, f=6, g=7, h=8) at suma_larga.c:3
```

En este punto es interesante ver el contenido de los registros `rdi, rsi, rdx, rcx, r8, r9` para verificar lo visto en teoría sobre la convención de llamada para C en x86-64:

```
(gdb) i r rdi rsi rdx rcx r8 r9
rdi            0x1      1
rsi            0x2      2
rdx            0x3      3
```

```
rcx            0x4      4
r8             0x5      5
r9             0x6      6
```

Vemos que efectivamente los primeros seis argumentos enteros fueron pasados en los registros mencionados en ese mismo orden.

El séptimo y el octavo argumento también son de tipo entero, por lo tanto según la convención de llamada son pasados utilizando la pila. Podemos verificarlo viendo el contenido de memoria:

```
(gdb) x/d $rsp+16
0x7fffffffe070: 7
(gdb) x/d $rsp+24
0x7fffffffe078: 8
```

También es interesante ver a que contenido está apuntando el registro `rsp`:

```
(gdb) x/1xg $rsp
0x7fffffffe060: 0x00007fffffffe080
```

Podemos verificar que este es el valor almacenado en el registro `rbp` luego del prólogo de la función `suma`. También podemos comprobar que el valor ubicado inmediatamente por encima en la pila corresponde a la dirección de retorno:

```
(gdb) x/1xg $rsp+8
0x7fffffffe068: 0x000000000040118f
```

dado que podemos ver que esta es la dirección de la instrucción siguiente al llamado a la función `suma`:

```
(gdb) disassemble main
Dump of assembler code for function main:
    0x0000000000401166 <+0>:   push   %rbp                  # Prólogo de la función main
    0x0000000000401167 <+1>:   mov    %rsp,%rbp

    0x000000000040116a <+4>:   push   $0x8
    0x000000000040116c <+6>:   push   $0x7
    0x000000000040116e <+8>:   mov    $0x6,%r9d             # 6º argumento
    0x0000000000401174 <+14>:  mov    $0x5,%r8d             # 5º argumento
    0x000000000040117a <+20>:  mov    $0x4,%ecx             # 4º argumento
    0x000000000040117f <+25>:  mov    $0x3,%edx             # 3º argumento
    0x0000000000401184 <+30>:  mov    $0x2,%esi             # 2º argumento
    0x0000000000401189 <+35>:  mov    $0x1,%edi             # 1º argumento
    0x000000000040118e <+40>:  call   0x401126 <suma>
    0x0000000000401193 <+45>:  add    $0x10,%rsp
    0x0000000000401197 <+49>:  mov    %eax,%esi
    0x0000000000401199 <+51>:  lea    0xe64(%rip),%rax      # 0x402004
    0x00000000004011a0 <+58>:  mov    %rax,%rdi
    0x00000000004011a3 <+61>:  mov    $0x0,%eax
    0x00000000004011a8 <+66>:  call   0x401030 <printf@plt>
    0x00000000004011ad <+71>:  mov    $0x0,%eax

    0x00000000004011b2 <+76>:  leave                        # Epílogo de la función suma
    0x00000000004011b3 <+77>:  ret
```

Asimismo podemos observar el área de construcción de variables temporales de la función `suma`:

```
(gdb) x/1xg $rbp-4
0x7fffffffe05c: 0xffffe08000000001
```

```
(gdb) x/1xg $rbp-8
0x7fffffffe058: 0x0000000100000002
(gdb) x/1xg $rbp-12
0x7fffffffe054: 0x0000000200000003
(gdb) x/1xg $rbp-16
0x7fffffffe050: 0x0000000300000004
(gdb) x/1xg $rbp-20
0x7fffffffe04c: 0x0000000400000005
(gdb) x/1xg $rbp-24
0x7fffffffe048: 0x0000000500000006
```

Notar que también podríamos haber usado el puntero `rsp`. Asimismo, podemos observar cómo se pasaron los argumentos séptimo y octavo por pila:

```
(gdb) x/1xg $rbp+16
0x7fffffffe070: 0x0000000000000007
(gdb) x/1xg $rbp+24
0x7fffffffe078: 0x0000000100000008
```

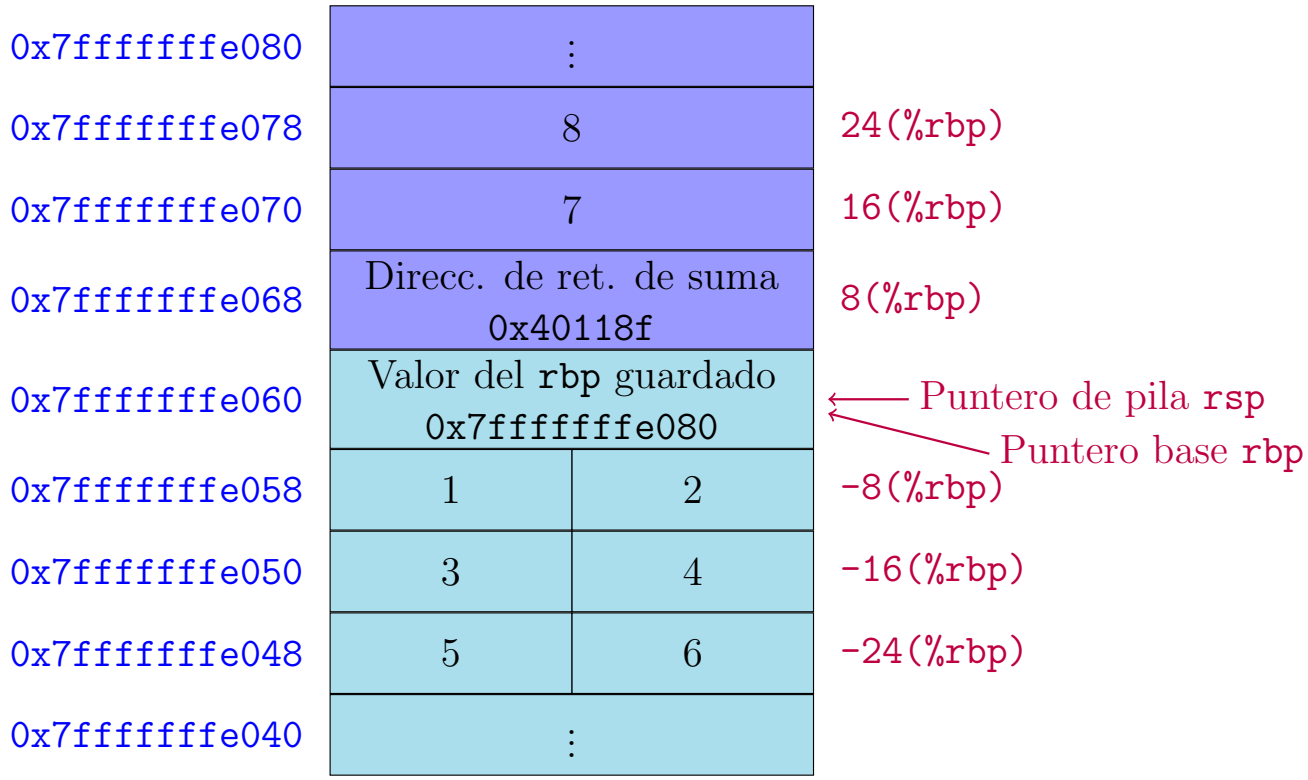
A continuación podemos ver el código equivalente de la función `suma` generado por el compilador:

```
(gdb) disassemble suma
Dump of assembler code for function suma:
   0x0000000000401126 <+0>:  push   %rbp                # Prólogo de la función suma
   0x0000000000401127 <+1>:  mov    %rsp,%rbp

   0x000000000040112a <+4>:  mov    %edi,-0x4(%rbp)     # Variable local, 1º argumento
   0x000000000040112d <+7>:  mov    %esi,-0x8(%rbp)     # Variable local, 2º argumento
   0x0000000000401130 <+10>: mov    %edx,-0xc(%rbp)     # Variable local, 3º argumento
   0x0000000000401133 <+13>: mov    %ecx,-0x10(%rbp)    # Variable local, 4º argumento
   0x0000000000401136 <+16>: mov    %r8d,-0x14(%rbp)    # Variable local, 5º argumento
   0x000000000040113a <+20>: mov    %r9d,-0x18(%rbp)    # Variable local, 6º argumento
   0x000000000040113e <+24>: mov    -0x4(%rbp),%edx
   0x0000000000401141 <+27>: mov    -0x8(%rbp),%eax
   0x0000000000401144 <+30>: add    %eax,%edx
   0x0000000000401146 <+32>: mov    -0xc(%rbp),%eax
   0x0000000000401149 <+35>: add    %eax,%edx
   0x000000000040114b <+37>: mov    -0x10(%rbp),%eax
   0x000000000040114e <+40>: add    %eax,%edx
   0x0000000000401150 <+42>: mov    -0x14(%rbp),%eax
   0x0000000000401153 <+45>: add    %eax,%edx
   0x0000000000401155 <+47>: mov    -0x18(%rbp),%eax
   0x0000000000401158 <+50>: add    %eax,%edx
   0x000000000040115a <+52>: mov    0x10(%rbp),%eax
   0x000000000040115d <+55>: add    %eax,%edx
   0x000000000040115f <+57>: mov    0x18(%rbp),%eax
   0x0000000000401162 <+60>: add    %edx,%eax

   0x0000000000401164 <+62>: pop    %rbp                # Epílogo de la función suma
   0x0000000000401165 <+63>: ret
```

El contenido esquemático de la pila se puede observar en la siguiente figura, luego de ejecutado el prólogo de la función `suma`:


0x7fffffffe080
0x7fffffffe078
0x7fffffffe070
0x7fffffffe068
0x7fffffffe060
0x7fffffffe058
0x7fffffffe050
0x7fffffffe048
0x7fffffffe040
8
7
Direcc. de ret. de suma
0x40118f
Valor del rbp guardado
0x7fffffffe080
1
2
3
4
5
6
24(%rbp)
16(%rbp)
8(%rbp)
Puntero de pila rsp
Puntero base rbp
-8(%rbp)
-16(%rbp)
-24(%rbp)